বাংলা সারসংক্ষেপ — Independent Kernel-Issue Audit

তারিখ: ২৬ অক্টোবর ২০২৬। **Evidence:** ব্যবহারকারীর ২৮৭-ফাইল ZIP, Kaggle discussion dump, live engine লগ, SGLang/vLLM upstream। **কোনো নতুন GPU run বা Kaggle submission করা হয়নি।**

প্রধান রায়

mikelou1-এর ৪০% থেকে ৬৬% লাফের নির্দিষ্ট "tiny kernel issue" প্রমাণিত নয়। তিনি GPU kernel, Kaggle notebook kernel, নাকি OS kernel বুঝিয়েছিলেন, কোথাও খোলসা করেননি। Kaggle-এ আগের একটি পোস্টে "running a kernel" বলতে notebook চালানো বোঝালেও সেটি চূড়ান্ত প্রমাণ নয়।

আপনাদের production-এ সবচেয়ে বেশি অগ্রাধিকার পাওয়ার মতো অমীমাংসিত correctness risk হলো SGLang #43567। আপনারা `extra_buffer_lazy`, ৬৮-slot Mamba pool ও ১৬×৪ live slots ব্যবহার করছেন। ZIP-এর `e14_sglang_patches.py`-তে #43567 backport রয়েছে, কিন্তু default apply list-এ সেটি নেই। Upstream #43567-এ cache-এর stale checkpoint-এ silent logit corruption নিশ্চিত হয়েছে। কিন্তু আপনার L4-তে trigger হওয়ার প্রমাণ এখনো নেই। Source: https://github.com/sgl-project/sglang/pull/43567

নতুন অনুসন্ধান: #43559 companion fix। একটি locked prefix-cache state reclaim না করতে পারলে Mamba allocator `AssertionError: Can not alloc mamba cache` দিয়ে server থামতে পারে। #43559 স্পষ্টভাবে #43567-এর ওপর নির্ভরশীল। শুধু একটি PR অন্ধভাবে বসাবেন না; exact-version compatible patch বা validated upgrade চাই। Source: https://github.com/sgl-project/sglang/pull/43559

R5-এর eval ৪৮.৬১ বনাম LB ২১.৩৯-এর মূল দৃশ্যমান কারণ coverage। আগের EVAL120 বিশ্লেষণে hidden coverage প্রায় ৪৯% ধরা হয়েছিল। সেই হিসাব অনুযায়ী আনুমানিক প্রত্যাশা ৪৮.৬১ × ০.৪৯ = **২৩.৮২**; LB-র সঙ্গে অবশিষ্ট ব্যবধান **২.৪৩ পয়েন্ট**। এই পার্থক্য একাই ২৭-পয়েন্ট kernel failure প্রমাণ করে না। তবে hidden task difficulty, ordering এবং task-output weighting ভিন্ন হতে পারে—এটি কেবল আনুমানিক decomposition।

আরেকটি গুরুত্বপূর্ণ rerun-only পার্থক্য production notebook-এর cell 2 line 235-এ আছে: `SGL_GIVE_UP_AT`-এর cutoff eval-এ **৭৫ মিনিট**, scored rerun-এ **২৪০ মিনিট**। SGLang স্বাভাবিকভাবে উঠে গেলে কোনো ক্ষতি নেই; কিন্তু বারবার launch fail হলে scored rerun-এ fallback-এর আগে অনেক বেশি সময় নষ্ট হতে পারে। এটিকে speculative issue বলছি, observed failure নয়।

GO/NO-GO

- **GO:** CPU-only log/branch checks, exact patch-state verification, সীমিত GPU smoke যেখানে target-only rollback আছে।
- **NO-GO:** #43567 আমাদের score-এর ২০+ পয়েন্ট নষ্ট করেছে—এ দাবি; mikelou1-এর secret fix শনাক্ত বলে দাবি; ২৫৬/১০২৪-token gate দিয়ে ৬০–৯০k-token correctness certified বলা; একতরফা untested source backport নিয়ে ১২ ঘণ্টার production run।

বিস্তারিত severity, cheapest test, fix ও প্রত্যাশিত leaderboard প্রভাব নিচের পূর্ণ independent research report-এ দেওয়া হলো। প্রত্যেক source-এর সম্পূর্ণ URL রাখা হয়েছে।

ARC-AGI-2 — Independent Kernel-Issue Research (26 October 2026)

Scope: `KERNEL_ISSUE_40_TO_66_RESEARCH_PACK_FOR_GPT_2026-10-26.zip` (287 files); Kaggle competitor discussion, archived our runs, source patches, notebooks, three 26-Oct live log excerpts, current upstream GitHub PRs/issues. **No GPU run or Kaggle submission made for this audit.**

Executive conclusions

1. **mikelou1's exact "tiny kernel issue" is UNKNOWN.** The competitor never disclosed the offending kernel, code, patch, engine, or model variant. The jump 40% → 66% is reported, not a causal experiment reproducible by

us. Notebook-kernel, GPU-kernel, and OS-kernel are all possible; the context more strongly motivates rerun/compute paths than OS-kernel failure, but no percentages/probabilities are justified.

2. **Our strongest reproducible upstream correctness candidate is SGLang PR #43567** on `extra_buffer_lazy` state invalidation. Upstream reproduced stale Mamba checkpoint restores with next-token full-vocab KL 0.6494482615 → 0 upon patch on GB300/Qwen3.5-0.8B. Our production 0.5.21 uses `extra_buffer_lazy`, four spare slots (68 total; 16×4 nominal) and massive reused prefixes; our backport contains the patch but `DEFAULT_NAMES=("44401","41872")` and the shipped production notebook does not activate `43567`. **Exact L4 effect not yet reproduced.** Source: https://github.com/sgl-project/sglang/pull/43567
3. **An adjacent fix matters:** SGLang #43559 (NOT the similarly numbered vLLM issue) protects lazy Mamba prefix locks under donation retry. Without it, slot pressure can terminate the engine with `AssertionError: Can not alloc mamba cache`; it explicitly depends on #43567. The package does not contain its backport. Source: https://github.com/sgl-project/sglang/pull/43559
4. **R5 eval 48.61 → LB 21.39 is mostly consistent with incomplete task coverage** rather than a demonstrated 27-point numerical bug. Independent run analysis estimated ~49% hidden coverage: 48.61×0.49=23.82, only 2.43 points above 21.39. This is an approximate decomposition, not an unbiased predicted score. E9's effective ratio 22.92/36.94=0.620. Task difficulty/order/hidden distribution and leaderboard variability remain confounds.
5. **Fresh notebook-specific potential failure:** `E15P_production_SGLang_Poetiq.ipynb` cell 2, line 235 sets `SGL_GIVE_UP_AT = min(NOTEBOOK_T0 + (240 if RERUN else 75) * 60, ADMIT_DEADLINE - 5 * 60)`. If SGLang repeatedly fails in scoring, the rerun may spend **up to 4h**, versus **75m** in eval, before vLLM fallback. This is a real branch difference, **not a confirmed failure** in any scored submission. Inspect each `RERUN` scoring log and engine start/fallback timestamps.
6. **New capacity warning:** 26-Oct production log has `kv_tokens=852152`, while configured 16 streams × `SGL_KV_PER_REQUEST=56000` = 896000. The gate's `SGL_MIN_KV=600000` can PASS despite being below this naive concurrency budget. This does NOT establish over-allocation, because 56k is admission planning, actual requests grow dynamically. Test long-context concurrent KV pressure and preemption, rather than interpreting startup gate as 16×56k guaranteed capacity.

Competitor evidence and vocabulary

Archived `01_competitor/A_mikelou1_all_posts_and_interpretations.md` and fresh Kaggle API topic-748082 dumps provide: October 11 ~25%, October 13 36%, October 16 CV ~60% versus score 40% with a “tiny kernel issue”, October 18 score 66% (~60% CV). He did not reveal engine, patch, quant, whether 27B or Flash-Next. On October 9 in thread 747515, “running a kernel” described Kaggle notebook execution. **This vocabulary clue is not proof of the October 16 referent.** Thread: https://www.kaggle.com/competitions/arc-prize-2026-arc-agi-2/discussion/748082 ; https://www.kaggle.com/competitions/arc-prize-2026-arc-agi-2/discussion/747515

Ranked suspects: cheapest decisive test, verified evidence, fix and LB impact

#	Cause and applicability	Strongest evidence	Cheapest decisive test	Safe remedy	LB effect
1	**Rerun notebook/time/branch divergence**. Directly applicable; E15P has rerun-dependent `PROFILE` and 240m/75m fallback cutoff.	User notebook cell 1 lines 17-45; cell 2 lines 2-3, 128-132, 235. Kaggle code reruns hidden data.	**CPU:** mock `RERUN=0/1`, trace all profile/time/paths; **logs:** scored `MODE`, first ready, failure retries, fallback, outputs written, attempted test task IDs per quarter-hour.	Fail-closed branch invariant: compare same code/config/hardware except dataset size and fixed 12h clock. Bound retries by elapsed time and automatic healthy-engine fallback; persist progress every batch.	**UNVERIFIED; potentially large if scoring branch actually stalls**. No numeric prediction.
2					

#	Cause and applicability	Strongest evidence	Cheapest decisive test	Safe remedy	LB effect
	SGLang #43567 stale Mamba lazy checkpoint. Strongly applicable to extra_buffer_lazy + near-full 68-slot pool + reused long prefixes.	https://github.com/sgl-project/sglang/pull/43567 ; upstream full-vocab KL 0.649→0.	**CPU:** patch-state check before/after; **short GPU:** warm/cold prefix KL under allocation pressure and 16 concurrent streams; compare unpatched vs verified backport, target-only and DFlash.	Apply the ZIP's two exact source hunks for 43567 **explicitly**; hash installed file, fail-close on unsuccessful patch; preferably use validated updated SGLang; avoid unsafe lazy cache if unable to patch.	**UNVERIFIED**, could recover silently wrong cached continuations.
3	**SGLang #43559 donation lock assertion**. Adjacent issue, cache/mamba slots under pressure; impact is crash/coverage.	https://github.com/sgl-project/sglang/pull/43559 ; verified red/green allocation regression, depends on #43567.	**CPU:** inspect installed lock/retry logic; **short GPU:** one/four request pool pressure reproduction under cached prefixes, inspect `Can not alloc mamba cache`.	Validated upstream PR #43567 + #43559-compatible build; do **not** blind-copy unified-cache patch to 0.5.21 legacy path. Operational fallback: fewer streams/more Mamba slots after measured memory checks, or disable radix cache.	**UNVERIFIED**, potentially large only if server dies or stalls.
4	**DFlash GDN verify state drift / concurrency**. SGLang DFlash6, >=12 streams is relevant, but exact 0.5.21 L4 failed not established.	https://github.com/sgl-project/sglang/issues/36548 ; upstream serial 0/100 wrong, c8 111/304 wrong, target-only 0/304, mitigations incompletely fix.	32+ deterministic request IDs, target-only vs DFlash, forced-reject, solo vs batch at 12/16 and 60k-prefix warm/cold; log first divergent recurrent layer and token IDs.	Disallow unvalidated DFlash; fallback target-only or independently validated MTP; fp32 SSM, disabled decode CUDA graph, width 4 are **mitigations, not proven fixes**.	**UNVERIFIED**, possibly substantial if correctness differs under real ARC concurrency.
5	**vLLM MTP + hybrid prefix caching**. Especially historical vLLM 0.19 path, not demonstrated fixed on exact model; 0.32 improvements are bug-specific.	https://github.com/vllm-project/vllm/issues/43559 ; https://github.com/vllm-project/vllm/issues/53912 . Note #43559 here is vLLM, **not** SGLang PR.	Long same-prefix multi-turn scoring; prefix cache ON/OFF, MTP ON/OFF, concurrency 1/16, compare full-token outputs and sampled distribution.	`--no-enable-prefix-caching` for correctness isolation; separately validate output quality and time.	**UNVERIFIED; old report title ~20% is not our expected +20%.**
6	**Mamba SSM fp16 versus fp32**. Model config float32; production override float16. Long 40–90k continuous generation is untested.	User checkpoint config and `03_our_data/C_ours_silent_accuracy_evidence.md`. No exact published regression for our stack verified.	Compare score (teacher-forced NLL/logprobs) on same 10k/40k/60k/90k stored continuations between fp16/fp32; A/A noise control.	`--mamba-ssm-dtype float32` only if measured gain justifies extra memory/admission reduction; else stay fp16.	**UNVERIFIED**.
7	**Mid-thinking EOS / parser loss**. E11-06 reports 10/44 mid-thinking EOS and 5/88 unfenced SGL answers versus	User `C_ours` and https://github.com/QwenLM/Qwen3.8/issues/214 (different long-context EOS	**CPU:** classify saved EOS token IDs/finish reasons; reconstruct failing prompts; compare reference EOS rank/	Safer parsing for raw code; explicit answer format; structured thinking-stop policy only after spec	**Observed small parser loss; future LB gain unverified.** Five recovered bare-code

#	Cause and applicability	Strongest evidence	Cheapest decisive test	Safe remedy	LB effect
	0/432 vLLM S1, but different prompts/ engines/caps.	incident; no deterministic repro).	probability at same token site on target-only; parse all saved unfenced programs.	compatibility test. Avoid assuming every EOS is kernel corruption.	programs did not pass train.
8	**FP8 KV scaling/ quant/different logits**. INT4 W4A16, FP8 KV, calibrated k/ v scale may differ.	User `C_ours`: checkpoint scales 0.0214–0.254 appear supported by source; runtime not measured.	Instrument `k_scale_float`, `v_scale_float` layer-by-layer and compare with checkpoint; compare selected logits with KV dtype `auto`.	Fix mapper/loader only if actual scale mismatch; retain measured correct path.	**UNVERIFIED**.
9	**L4 PCIe/TP4 throughput bottleneck and GCSFUSE cold startup**. Strong performance risk; not silent wrong logits by itself.	User TP4 logs show custom all-reduce disabled, SHM NCCL, ~522s engine ready on 26-Oct and historical ~1197s; https://docs.nvidia.com/deeplearning/nccl/user-guide/docs/troubleshooting/gpu_troubleshooting.html	Existing logs: split stage/copy/JIT/ graphs/launch; CPU filesystem staging timing; short GPU 8KB–1MB NCCL benchmark/TP4 vs 2×TP2.	Prestage weights to `/dev/shm` with verified hash; topology-aware GPU pairing; bounded model startup; avoid repeated cold starts; CPU/GPU performance traces.	**UNVERIFIED; coverage-mediated only.**
10	**OS kernel-level race/resource issue**. Least evidence specific to our run.	No direct L4, driver, kernel-specific fault established in the pack.	`uname -r`, driver/ PCIe topology, `dmesg` if allowed, process exit codes, shared memory, cgroups/ ulimits; request-level failing signatures.	Only after matching a concrete error/ host difference.	**UNVERIFIED**.

Important extra source-level concerns beyond previous summaries

- `e14_sglang_patches.py`: exact #43567 patch hunks are present at lines 55–58 and registered at lines 196–199, but `DEFAULT_NAMES = ("44401", "41872")` at line 33. Production notebooks cannot be assumed patched because a source file contains the hunk. A runtime explicit-apply report with installed-file SHA is mandatory.
- **#43567 + #43559 are coupled**, and SGLang PR #43387 concerns a different `no_buffer` + speculative truncation failure. #43387 should **not** be conflated with `extra_buffer_lazy` fix. https://github.com/sgl-project/sglang/pull/43387
- `--enable-mixed-chunk` + extra-buffer was independently reported to corrupt cache checkpoints, but our production command does not list it; **not applicable unless it is actually on**. https://github.com/sgl-project/sglang/issues/39342
- vLLM 0.32.0 includes a **specific** fix for DFlash + cache block-layout corruption (#60174/#55601); this does not prove the separate vLLM MTP issue #53912 fixed. https://github.com/vllm-project/vllm/issues/60174 ; https://github.com/vllm-project/vllm/pull/55601
- The 26-Oct VMS log excerpts show kernel selection `MarlinFP8ScaledMMLinearKernel` (not assumed Triton) and variants `ok:true` / `ok:false`, but lines are truncated and are **not complete gate/outcome artifacts**. Do not assign causes to those statuses from incomplete excerpts.
- At temperatures >0, raw sampled token-stream equality is not a reliable speculative-correctness invariant. Use greedy fixed same-checkpoint comparisons for deterministic checks, plus statistical distribution/ARC-task checks at T=1.0.

Exact minimal code / validations

A. Fail-closed patch activation (after install, before SGLang import)

```
from pathlib import Path
import e14_sglang_patches as patches

pkg = Path('/tmp/sglang-venv/lib/python3.12/site-packages/sglang')
report = patches.apply(pkg, names=('44401', '41872', '43567'))
print(report)
post = patches.check(pkg, names=('43567',))
assert post['patches']['43567']['state'] == 'applied', post
```

Important: `patches.check`/`patches.apply` return schema and `pkg_dir` should first be confirmed against this ZIP's implementation and actual installed package. The snippet above is an integration pattern to adapt, not an executed proof. Do not use a silent `try: ... except: pass`. For #43559, validated upstream port or a separately tested v0.5.22 upgrade is preferable to invented old-version text substitutions.

B. Re-run branch invariant (CPU)

```
for rerun in (False, True):
    give_up_minutes = 240 if rerun else 75
    print('rerun', rerun, 'fallback_cutoff_minutes', give_up_minutes)
# If the scoring launch path fails repeatedly, this difference can lose 165 min.
```

Test: force (mock) two failed 30-minute startups, preserve resulting produced task count and score-proxy. A healthy engine that starts in 522 sec is **not** affected by this cutoff. Compare this condition explicitly, do not claim it happened in an actual submission.

C. Cache restore parity

For identical tokenizer IDs and model: (1) cold score 55–60k prefix; (2) warm cache using near-identical shared-prefix trace; (3) score the same next 2k token IDs; (4) compare KL/nll to cold-vs-cold control, log actual prefix hits; (5) repeat at batch 1, 12 and 16 and near-full 68 slots. Require complete returned logits, not just first 256 greedy tokens.

D. Evaluated LB gap

R5 `eval_task_norm=48.61`, observed `LB=21.39`, modelled effective hidden coverage about 49%. `48.61*0.49=23.8189`; residual difference `2.4289` LB points. User estimated same-notebook LB noise SD ~1.5 points, so residual alone does not establish a silent kernel bug. Crucially the evaluation and leaderboard use different tasks/order, so the product model is **illustrative**, not an unbiased estimator.

E. One-pass 0-GPU checklist

Check all scored rerun `MODE`, `PROFILE`, model path, hash, launch attempts, fallback, first request timestamp, task_count, verified programs committed, invalid/empty/identity submissions, hidden file path, `KAGGLE_IS_COMPETITION_RERUN`, completion JSON count and outstanding futures. If those are healthy, investigate GDN state and cache with targeted short GPU tests; if missing, fix orchestration before costly numerical experiments.

GO / NO-GO

- **GO:** Read-only evidence gathering, no-GPU rerun-path diff, patch-static checks, log normalization, short *diagnostic* GPU tests with target-only rollback.
- **NO-GO:** Claim that mikelou1's issue is identified; claim that #43567 explains our 20+ LB shortfall; deploy an unverified #43567-only backport as full fix; treat short-token speculative parity as a 60k-token accuracy certificate; blindly switch long production to fp32 SSM without capacity testing.

Sources (full literal URLs):

- https://www.kaggle.com/competitions/arc-prize-2026-arc-agi-2/discussion/748082
- https://www.kaggle.com/competitions/arc-prize-2026-arc-agi-2/discussion/747515
- https://www.kaggle.com/competitions/arc-prize-2026-arc-agi-2/overview/upgraded-accelerators
- https://github.com/sgl-project/sglang/pull/43567
- https://github.com/sgl-project/sglang/pull/43559
- https://github.com/sgl-project/sglang/pull/43387
- https://github.com/sgl-project/sglang/pull/43029
- https://github.com/sgl-project/sglang/issues/36548
- https://github.com/sgl-project/sglang/issues/39342
- https://github.com/sgl-project/sglang/issues/36880
- https://github.com/sgl-project/sglang/issues/36889
- https://github.com/vllm-project/vllm/issues/43559
- https://github.com/vllm-project/vllm/issues/53912
- https://github.com/vllm-project/vllm/issues/60174
- https://github.com/vllm-project/vllm/pull/55601
- https://github.com/QwenLM/Qwen3.8/issues/214
- https://docs.nvidia.com/deeplearning/nccl/user-guide/docs/troubleshooting/gpu_troubleshooting.html